

Fake User Generator

Task 6 | PostgreSQL + Python / Flask

1. Overview

A web application that generates deterministic fake user contact data. All generation logic lives exclusively in PostgreSQL stored procedures — Python/Flask is just a read-only client that calls them and renders HTML.

- Two locales supported: **English / USA** and **German / Germany**
- User picks a **seed** and a **batch index** — same inputs always return same data
- 10 users generated per batch; navigate forward/backward freely

2. Tech Stack

Component	Role
PostgreSQL 17	All generation logic — stored procedures only
Python 3 + Flask	Web server, renders HTML templates
psycopg2	Python ↔ PostgreSQL connector
HTML / CSS	Frontend UI

3. Database Schema

One extensible table — adding a new locale requires only inserting rows, no schema changes:

```
fake.lookup ( id SERIAL, locale VARCHAR(10), category VARCHAR(50), value TEXT )
locale → 'en-US' | 'de-DE' | 'any' category → 'first_male' | 'first_female' | 'last'
| 'middle_male' | 'title_male' | 'title_female' | 'street_name' | 'street_suffix'
| 'city' | 'eye_color' | 'email_domain'
```

- ~150 first names per gender per locale, ~500 last names → 150 000+ unique full names
- 50 cities per locale with state and ZIP pattern (stored as JSON in value)
- 100+ street names, 6 eye colors, 20 email domains

4. Core RNG — fake.rng(seed, idx, field)

Hash-based deterministic pseudo-random number generator. No RANDOM(), no sequences.

```
key = seed || chr(1) || idx || chr(1) || field raw = abs( hashtext(key) ) -- int in
[0, 2^31) result = (raw % 1_000_000_000) / 1e9 -- float in [0, 1)
```

- Declared **IMMUTABLE** — PostgreSQL can cache / inline it
- Each (seed, idx, field) triple is an independent, reproducible stream
- chr(1) separator prevents collisions like '1|23' vs '12|3'

5. Stored Procedures — Public API

Procedure	Returns	Description
<code>fake.rng(seed, idx, field)</code>	float8	Base deterministic RNG
<code>fake.rng_int(..., lo, hi)</code>	int	Uniform integer in [lo, hi]
<code>fake.rng_normal(..., mean, s)</code>	float8	Normal dist. via Box-Muller
<code>fake.generate_name(locale, seed, idx)</code>	text	Full name, optional title/middle
<code>fake.generate_address(locale, seed, idx)</code>	text	Postal address, 3 variants
<code>fake.generate_coordinates(seed, idx)</code>	text	Lat/lon uniform on sphere
<code>fake.generate_physical_attributes(...)</code>	jsonb	Height, weight, age, eye color
<code>fake.generate_phone(locale, seed, idx)</code>	text	Phone, 4 format variants
<code>fake.generate_email(locale, seed, idx)</code>	text	Email, 5 pattern variants
<code>fake.generate_user(locale, seed, idx)</code>	jsonb	Complete user record
<code>fake.generate_users(locale, seed, batch, n)</code>	TABLE	Batch of n users
<code>fake.benchmark(locale, seed, n_batches)</code>	float8	Throughput in users/sec

6. Key Algorithms

Normal Distribution — Box-Muller Transform

```
U1 = max( rng(..., field), 1e-10 ) -- avoid ln(0)
U2 = rng(..., field+1)
Z = sqrt( -2 * ln(U1) ) * cos( 2*pi * U2 ) -- N(0,1)
result = mean + stddev * Z
```

- Height: male N(178 cm, 7) female N(165 cm, 6) clamped [140, 220]
- Weight: male N(82 kg, 14) female N(67 kg, 12) clamped [40, 180]
- Age: N(38, 15) clamped [18, 90]

Sphere-Uniform Geolocation

Naive approach (uniform lat in [−90, 90]) clusters points near the poles because equal angular bands cover less surface area there. The correct method uses the inverse CDF:

```
lat = degrees( arcsin( 2*U1 - 1 ) ) -- uniform on sphere surface
lon = 360 * U2 - 180
-- uniform in [-180, 180)
```

Reproducibility

```
global_index = batch_index * batch_size + position_in_batch
fake.rng( seed, global_index, field ) -- always the same value
```

7. Generated Fields per User

Field	Details
Full name	Title (25%) + First + Middle (40%) + Last
Address	Number + Street + Suffix + City + State + ZIP, 3 layout variants
Geolocation	Lat / lon — uniform on sphere (inverse CDF method)

Field	Details
Physical	Height & weight (normal dist.), age (normal, clamped), eye color (discrete)
Phone	4 format variants per locale — e.g. +1 (555) 234-5678 or 0175 1234567
Email	5 patterns: firstname.last, f.last, first_last, last.first, firstlast

8. How to Run

```
# 1. Create database createdb fakegen # 2. Load schema, procedures, data psql fakegen
-f db/01_schema.sql psql fakegen -f db/02_procedures.sql psql fakegen -f
db/03_seed_data.sql # 3. Install Python deps and start pip install -r
requirements.txt python app.py # → http://localhost:5000
```

9. Benchmark

- Browser: **GET /api/benchmark?locale=en-US&seed=42&n_batches=200**
- SQL: **SELECT fake.benchmark('en-US', 42, 500);**
- Returns users/second — typically several thousand on a local machine